```
    public string PhoneNumber { get; set; }
}
```

❶ Adds this using statement to use the validation attributes

❷ The API takes a UserModel parameter and binds it to the request body.

❸ Values marked Required must be provided.

❹ The StringLengthAttribute sets the maximum length for the property.

❺ Customizes the name used to describe the property

❻ Validates that the value of Email may be a valid email address

❼ Validates that the value of PhoneNumber has a valid telephone number format

Suddenly, your parameter type, which was sparse on details, contains a wealth of information. You've specified that the `FirstName` property should always be provided; that it should have a maximum length of 100 characters; and that when it's referred to (in error messages, for example), it should be called `"Your name"` instead of `"FirstName"`.

The great thing about these attributes is that they clearly declare the *expected* state of an instance of the type. By looking at these attributes, you know what the properties will contain, or at least *should* contain. Then you can then write code after model binding to confirm that the bound parameter is valid, as you'll see in section 7.10.3.

You've got a plethora of attributes to choose among when you apply `DataAnnotations` to your types. I've listed some of the common ones here, but you can find more in the `System.ComponentModel.DataAnnotations` namespace. For a more complete list, I recommend using IntelliSense in your IDE or checking the documentation at http://mng.bz/e1Mv.

- `[CreditCard]`—Validates that a property has a valid credit card format